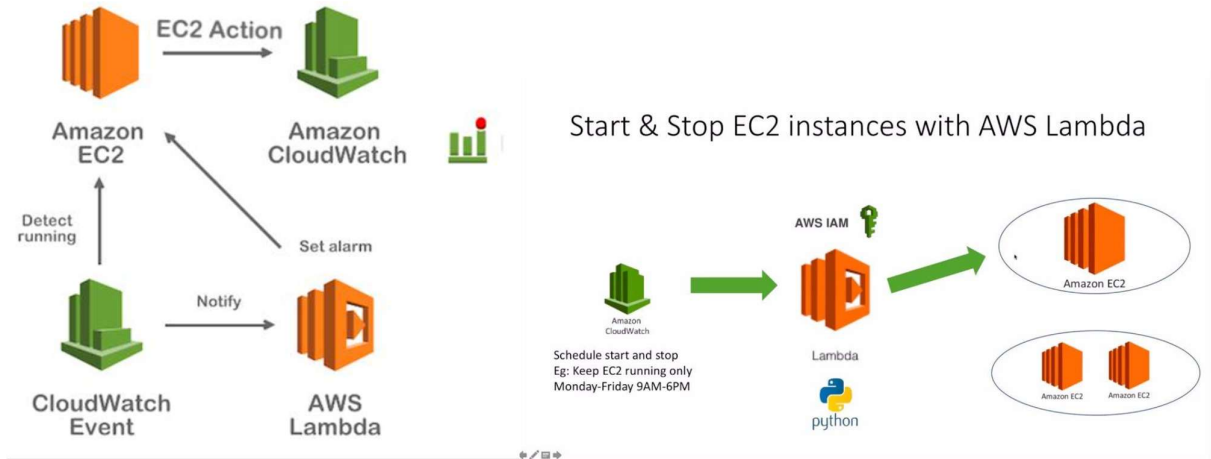


Name : Vivek Chaudhari

Project -

EcoSave : Automated AWS cost reduction using Lambda



Amazon EC2 → Launch EC2 instance for testing

AWS Lambda → Create function to automate logic

Amazon CloudWatch → Monitor EC2 CPU usage

Lambda → Fetch all running EC2 instances

Lambda → Check CPU utilization of each instance

Decision → If CPU < 10% (idle)

Action → Stop EC2 instance automatically

CloudWatch → Trigger Lambda on schedule

Output → Idle instances stopped successfully

Result → AWS cost optimized 💰

Step 1 : Create EC2 Instance :

Instances (4) Info										
Saved filter sets										
Choose filter set										
Find Instance by attribute or tag (case-sensitive)										
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐		i-02d1230dd76f53191	Stopped	t3.micro	–	View alarms +	ap-south-1b	–	–	–
☐		i-09f8e3519d7f964fa	Stopped	t3.micro	–	View alarms +	ap-south-1a	–	–	–
☐	cost-optimize...	i-0c15e5a1195e1e1b2	Stopped	t3.micro	–	View alarms +	ap-south-1a	–	–	–
☐		i-009ec48a44d36d28e	Running	t3.micro	Initializing	View alarms +	ap-south-1a	ec2-13-233-55-35.ap-s...	13.233.55.35	–

Step 2 : Cerate IAM Role and Attach Policy ;

cost-optimize-function-role-jniggt65 [Info](#) [Delete](#) [Edit](#)

Summary

Creation date
April 23, 2026, 14:24 (UTC+05:30)

Last activity
-

ARN
[arn:aws:iam:216153283160:role/service-role/cost-optimize-function-role-jniggt65](#)

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (3) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type
All types

Policy name	Type	Attached entities
AmazonEC2FullAccess	AWS managed	2
AmazonEC2ReadOnlyAccess	AWS managed	2
AWSLambdaBasicExecutionRole-b9a5be77-096b-4f...	Customer managed	1

Step 3 : Create Lambda Function and attach role policy

cost-optimize-function [Throttle](#) [Copy ARN](#) [Actions](#)

Function overview [Info](#) [Export to Infrastructure Composer](#) [Download](#)

[Diagram](#) [Template](#)

cost-optimize-function

[+ Add trigger](#) [+ Add destination](#)

Code | **Test** | **Monitor** | **Configuration** | **Aliases** | **Versions**

Monitor [Info](#) [Investigate with AI - new](#) [Filter metrics by](#) [Function](#) [View CloudWatch logs](#)

[Alarm recommendations](#) [Explore related](#)

Step 4 : Code Optimize :

```
1 from datetime import datetime, timedelta
2 import boto3
3
4 ec2 = boto3.client('ec2')
5 cloudwatch = boto3.client('cloudwatch')
6
7 def lambda_handler(event, context):
8     stopped = []
9
10    instances = ec2.describe_instances()
11
12    for reservation in instances['Reservations']:
13        for instance in reservation['Instances']:
14            instance_id = instance['InstanceId']
15
16            response = cloudwatch.get_metric_statistics(
17                Namespace='AWS/EC2',
18                MetricName='CPUUtilization',
19                Dimensions=[('Name': 'InstanceId', 'Value': instance_id)],
20                StartTime=datetime.utcnow() - timedelta(minutes=60),
21                EndTime=datetime.utcnow(),
22                Period=300,
23                Statistics=['Average']
24            )
25
26            if response['Datapoints']:
27                cpu = response['Datapoints'][0]['Average']
28
29                print(f"{instance_id} CPU: {cpu}")
30
31                if cpu < 10:
32                    ec2.stop_instances(InstanceIds=[instance_id])
33                    stopped.append(instance_id)
34
35    return {
36        "stopped_instances": stopped
37    }
```

Step 5 : Final Output :

Log events		Actions	Start tailing
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns			
Filter events - press enter to search		Clear	1m 30m 1h 12h Custom UTC timezone
Timestamp	Message		
No older events at this moment. Retry			
2026-04-23T09:05:22.935Z	INIT_START Runtime Version: python3.14.v39 Runtime Version ARN: arn:aws:lambda:ap-south-1:runtime06dae9ab05ccf0506ef62a6c3f8c3976f2962ec872732763a94c0841151809		
2026-04-23T09:05:23.488Z	START RequestId: a3c63d8e-2441-4b36-b83f-e0b6c8fae34b Version: \$LATEST		
2026-04-23T09:05:24.041Z	I-02d1238d576f53f91 CPU: 0.3488025359994611		
2026-04-23T09:05:24.461Z	I-09f8e3519d7f964fa CPU: 0.26495782568499324		
2026-04-23T09:05:24.868Z	I-0c15fa1195e1e1d2 CPU: 0.14328368474972747		
2026-04-23T09:05:25.239Z	END RequestId: a3c63d8e-2441-4b36-b83f-e0b6c8fae34b		
2026-04-23T09:05:25.248Z	REPORT RequestId: a3c63d8e-2441-4b36-b83f-e0b6c8fae34b Duration: 1749.53 ms Billed Duration: 2293 ms Memory Size: 128 MB Max Memory Used: 104 MB Init Duration: 543.19 ms		
No newer events at this moment. Auto retry paused . Resume			

Summary of Project :

The **EcoSave** project is an automated cost optimization system built on **Amazon Web Services** that helps reduce unnecessary cloud expenses by intelligently managing compute resources. It uses **AWS Lambda** to execute automation logic, **Amazon CloudWatch** to track CPU utilization, and **Amazon EC2** instances as the resources being optimized. The system periodically checks the CPU usage of running instances, and if any instance is found to be underutilized (below a defined threshold), it automatically stops it. This eliminates manual monitoring, improves resource efficiency, and significantly reduces AWS costs, making it a practical and scalable solution for cloud cost management.